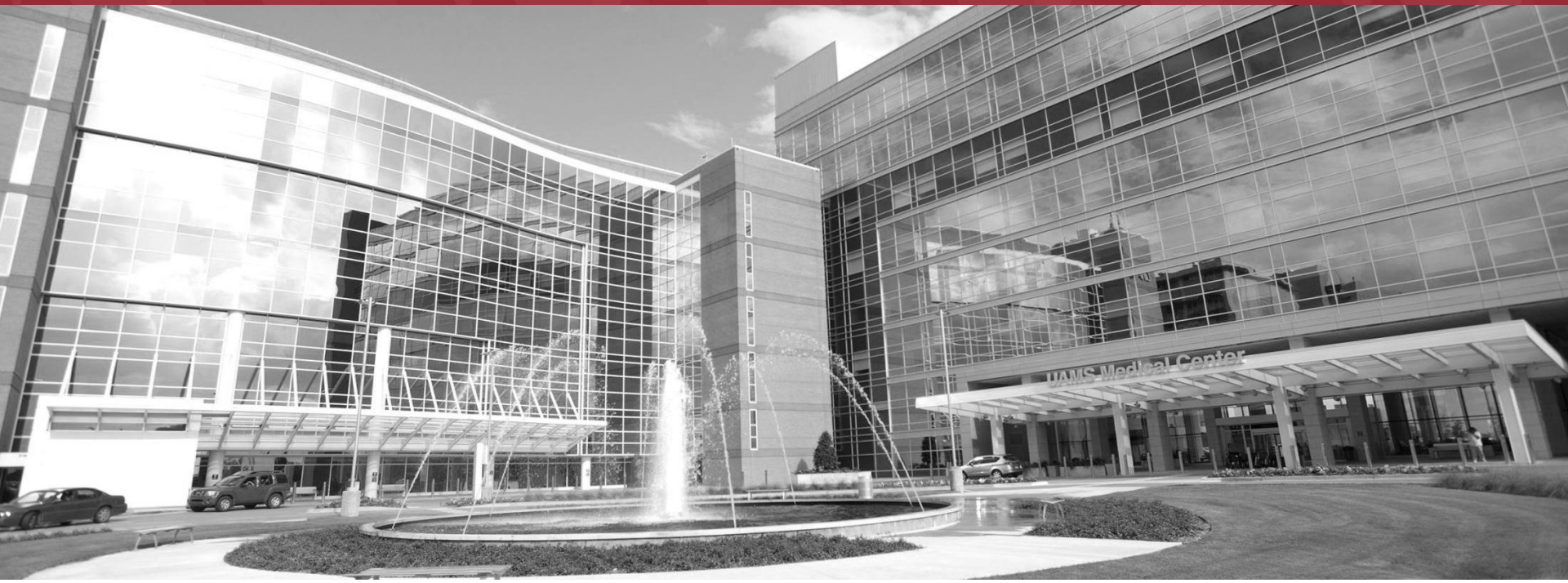


UAMS College of Medicine

Arkansas INBRE
IDeA Network of Biomedical Research Excellence

Workshop: Introduction to R

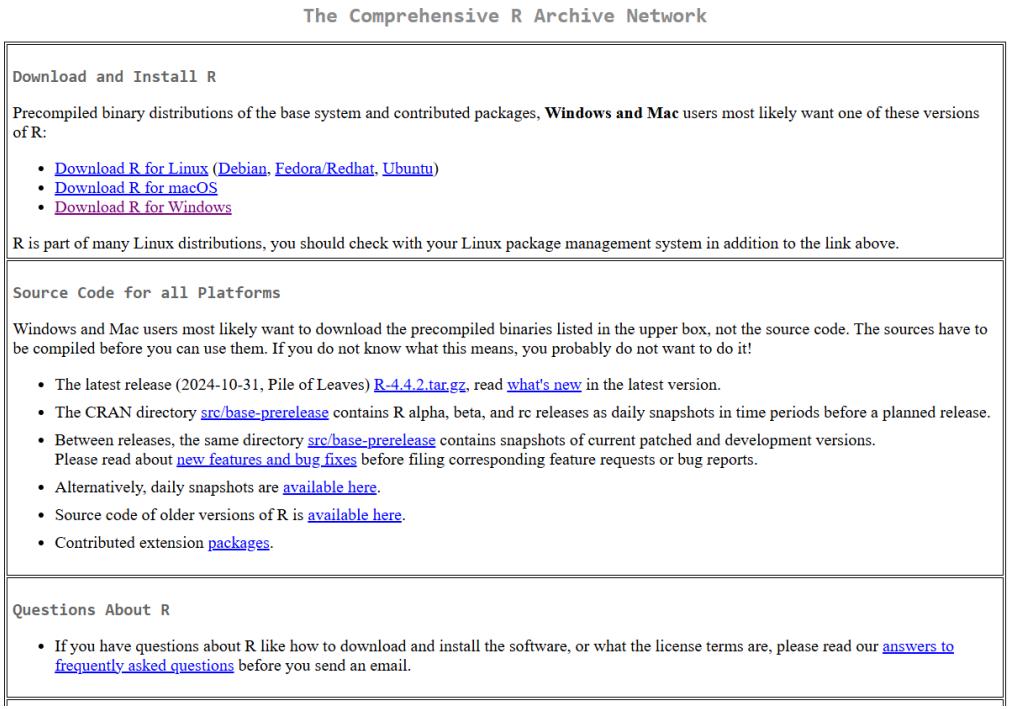
Delivered by the Data Science Core

Presenter: Yasir Rahmatallah, PhD
Associate Professor
Department of Biomedical Informatics (DBMI)
University of Arkansas for Medical Sciences

Date: October 2026

R Installation

- R is a free software environment for statistical computing and graphics.
- Download R from https://www.r-project.org and install it.
- Packages for general purposes are available from The Comprehensive R Archive Network (***CRAN***) repository.
- Packages for the processing and analysis of omics data are available from the ***Bioconductor*** repository.



The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- Download R for Linux (Debian, Fedora/Redhat, Ubuntu)
- Download R for macOS
- Download R for Windows

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

Source Code for all Platforms

Windows and Mac users most likely want to download the precompiled binaries listed in the upper box, not the source code. The sources have to be compiled before you can use them. If you do not know what this means, you probably do not want to do it!

- The latest release (2024-10-31, Pile of Leaves) R-4.4.2.tar.gz, read what's new in the latest version.
- The CRAN directory src/base-prerelease contains R alpha, beta, and rc releases as daily snapshots in time periods before a planned release.
- Between releases, the same directory src/base-prerelease contains snapshots of current patched and development versions. Please read about new features and bug fixes before filing corresponding feature requests or bug reports.
- Alternatively, daily snapshots are available here.
- Source code of older versions of R is available here.
- Contributed extension packages.

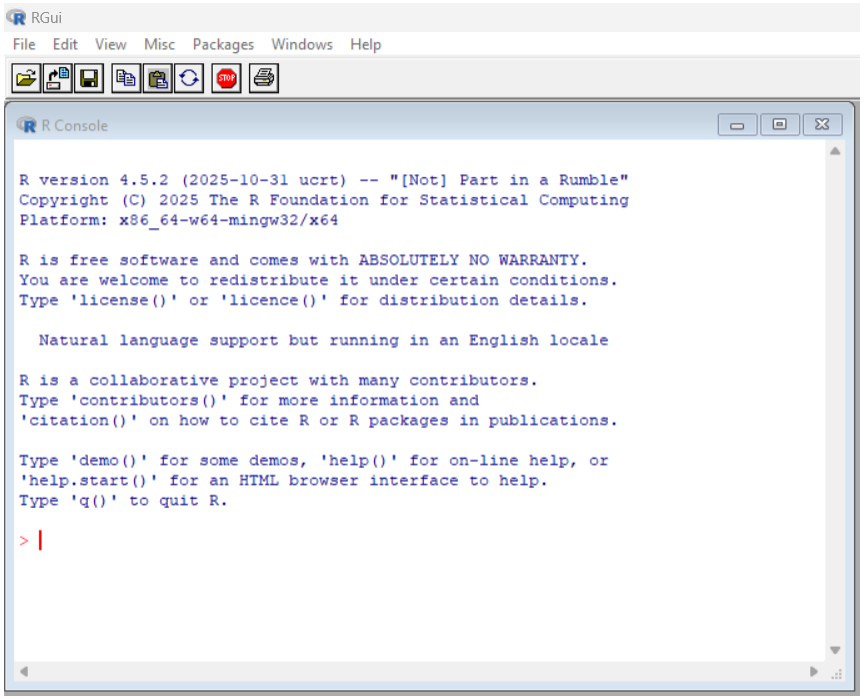
Questions About R

- If you have questions about R like how to download and install the software, or what the license terms are, please read our answers to frequently asked questions before you send an email.

- After installing R, you can use the basic R user interface to run command lines, write code lines and/or execute them, save/load code, ...
- Other graphical user interfaces with additional features, such as ***Rstudio***, (recently renamed to posit) are also available for free.
- ***Rstudio*** requires installing R first.
- ***Rstudio*** shows the available data objects in workspace and the created plots in real time within its GUI. Regular R installation lacks this property.
- To instal Rstudio (posit), check https://posit.co/downloads
- In this workshop, we will use the basic/regular R available from the R project. Anyone is welcomed to use Rstudio instead if it feels more convenient.

Package Installation

- R has a command line interface.
- Packages (or libraries) can be easily installed from the Comprehensive R Archive Network (CRAN) using 'Install packages' from the 'Packages' pop-down menu of the GUI. Packages can also be installed using command `install.packages("package-name")`
- Packages can be installed from Bioconductor using package ***BiocManager***, but you need to install package BiocManager from CRAN first using `install.packages("BiocManager")`
- To use some package/library, load it to your R session using `library(lib_name)` then you can use any command from the package. Alternatively, you can use a command without loading its package by listing the package name first, followed by double colons ::, followed by the command (as shown above).



Bioconductor

- Bioconductor is an open-source software project that provide tools for the analysis of a variety types of data, including high throughput genomics datasets. It is based primarily on the R programming language.
- Bioconductor has 4 categories of packages: (1) software packages (2) annotation packages (3) experiment data packages (4) workflow packages. We will mostly use the first two!
- R package ***BiocManager*** (download from CRAN using command `install.packages`) is the tool to install Bioconductor packages (allows installing a specific version). For example,

```
> if (!requireNamespace("BiocManager", quietly = TRUE))
install.packages("BiocManager")
> BiocManager::install(version = "3.22") # install Bioconductor

# command requireNamespace will return TRUE if package BiocManager
has been already installed. Otherwise, it returns FALSE and the first
line start installing package BiocManager. Notice the negation
operator (!).
# package_name::command(parameters …) allows you to use commands from
a specific package even without loading the package first.

> BiocManager::install("package_name") # install a specific
Bioconductor package
```

- For more details, check https://www.bioconductor.org/install/

Notes

- You can execute one line of code in the main R console.
- To execute multiple lines of code, save code, or open it later, you need to create R scripts. From the ‘File’ menu, select ‘new script’.
- To open a saved script, select ‘open script’ from the ‘File’ menu.
- Any created data objects or functions will be saved in a temporary ***Workspace*** that is lost when you terminate the current session (if not saved). To save all data objects currently in the Workspace, select ‘Save Workspace’ from the ‘File’ menu. Saved Workspace can be loaded to the current R session by selecting ‘Load Workspace’ from the ‘File’ menu. Alternatively, commands **save** and **load** can be used for the same purpose (will use them in future examples).
- Delete data objects from workspace using command `rm(object_name)`
- Print the names of object already available in workspace using command `ls()`.

Data Structures in R

- Numeric/character variables and vectors
- Matrices and Arrays
- Lists
- Data frames
- Factors

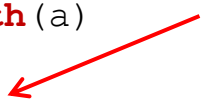
In addition to the basic data structure (object) types defined in R, multiple packages define their own data structures to facilitate special tasks (e.g. storing gene expression or nucleotide sequence data).

Numeric and character variables/vectors

```
Comments in R code are preceded
with # mark

> a <- 5
# or equivalently a = 5
> class(a)
[1] "numeric"
> length(a)
[1] 1
> b <- c(1,2,3,4,5)
# or equivalently b <- c(1:5)
> class(b)
[1] "numeric"
> length(b)
[1] 5 # numeric vector of length 5
> c <- "Hello"
> class(c)
[1] "character"
> length(c)
[1] 1
```

concatenation

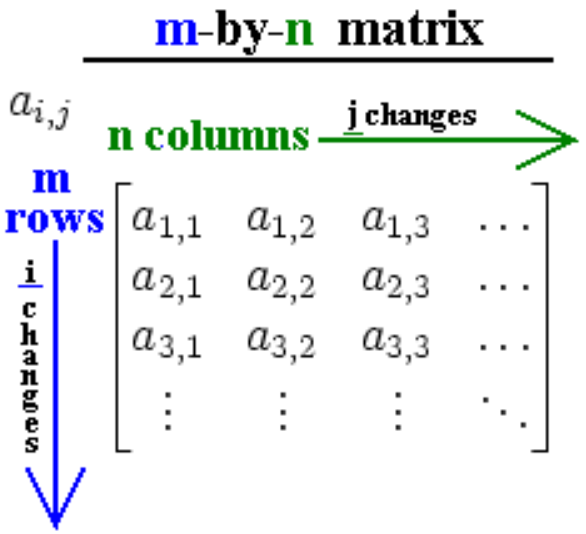


```
> d <- c("green","red","blue")
> class(d)
[1] "character"
> length(d)
[1] 3
# character vector of length 3

> a * 5
[1] 25
> a / 2
[1] 2.5
> a^3
[1] 125
> a * b
[1]  5 10 15 20 25
# element-wise operation
> b * b
[1]  1  4  9 16 25
```

Matrices

```
> mat1 <- matrix(data=c(1:9), nrow=3, ncol=3, byrow=TRUE,
dimnames=list(c("r1","r2","r3"), c("c1","c2","c3")))
> mat1
   c1 c2 c3
r1  1  2  3
r2  4  5  6
r3  7  8  9
> class(mat1)
[1] "matrix" "array"
> dim(mat1)
[1] 3 3
> nrow(mat1)
[1] 3
> ncol(mat1)
[1] 3
> length(mat1)
[1] 9
> rownames(mat1)
[1] "r1" "r2" "r3"
> colnames(mat1)
[1] "c1" "c2" "c3"
> which(mat1 < 3)
[1] 1 4
# command which returns the indices of the TRUE elements in a logical vector. It
searches the matrix column-wise
```



Matrix Indexing & Simple Functions

```
> mat1[2,1]
[1] 4
> mat1["r2","c1"]
[1] 4
> mat1[1,]
c1 c2 c3
 1  2  3
> mat1[,2]
r1 r2 r3
 2  5  8
> mat1[,"c2"]
r1 r2 r3
 2  5  8
> mat1[1:2,2:3]
   c2 c3
r1  2  3
r2  5  6
```

```
> sum(mat1)
[1] 45
> mean(mat1)
[1] 5
> median(mat1)
[1] 5
> rowSums(mat1)
r1 r2 r3
 6 15 24
> colSums(mat1)
c1 c2 c3
12 15 18
> rowMeans(mat1)
r1 r2 r3
 2  5  8
> colMeans(mat1)
c1 c2 c3
 4  5  6
```

```
> mat1 == 4
      c1    c2    c3
r1 FALSE FALSE FALSE
r2  TRUE FALSE FALSE
r3 FALSE FALSE FALSE
> which(mat1 == 4)
[1] 2
> mat1 > 5
      c1    c2    c3
r1 FALSE FALSE FALSE
r2 FALSE FALSE  TRUE
r3  TRUE  TRUE  TRUE
> which(mat1 > 5)
[1] 3 6 8 9
```

Matrix operations

```
> mat2 <- mat1 * 2
> mat2
   c1 c2 c3
r1  2  4  6
r2  8 10 12
r3 14 16 18
> mat1 * mat2
    c1   c2   c3
r1   2    8   18
r2  32   50   72
r3  98  128  162
> mat1 %*% mat2
     c1   c2   c3
r1   60   72   84
r2  132  162  192
r3  204  252  300
```

element-wise multiplication

matrix-wise multiplication

```
> mat1 + mat2
   c1 c2 c3
r1  3  6  9
r2 12 15 18
r3 21 24 27
> mat1 - mat2
   c1 c2 c3
r1 -1 -2 -3
r2 -4 -5 -6
r3 -7 -8 -9
> mat2 / mat1
   c1 c2 c3
r1  2  2  2
r2  2  2  2
r3  2  2  2
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \times \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \\ 14 & 16 & 18 \end{bmatrix} = \begin{bmatrix} 1\times 2 + 2\times 8 + 3\times 14 = 60 & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$$

Character Matrix

```
> mat3 <- matrix(c("one","two","three","four","five","six"),
nrow=2, ncol=3, byrow=FALSE)
# The default byrow=FALSE will be used if not specified
> mat3
     [,1]  [,2]    [,3]
[1,] "one" "three" "five"
[2,] "two" "four"  "six"
> class(mat3)
[1] "matrix" "array"
> dim(mat3)
[1] 2 3
> mat3[,3]
[1] "five" "six"
> which(mat3 == "five")
[1] 5
# command which returns the indices of the TRUE elements in a
logical vector. Matrices are checked column-wise.
```

Arrays

- Vectors are 1-dimensional arrays.
- Matrices are 2-dimensional arrays.

```
> z <- array(data=c(1:12),
dim=c(2,2,3),
dimnames=list(c("r1","r2"),
c("c1","c2"), c("m1","m2","m3")))
> z
, , m1

   c1 c2
r1  1  3
r2  2  4

, , m2

   c1 c2
r1  5  7
r2  6  8

, , m3

   c1 c2
r1  9 11
r2 10 12

> z[,,1]
   c1 c2
r1  1  3
r2  2  4
> z[1,,]
   m1 m2 m3
c1  1  5  9
c2  3  7 11
> z[1,1,]
m1 m2 m3
 1  5  9
```

List Object

```
# a list object is a container to combine multiple objects of different types

> mylist <- list(a, b, c, d, mat1)
> mylist
[[1]]
[1] 5

[[2]]
[1] 1 2 3 4 5

[[3]]
[1] "Hello"

[[4]]
[1] "green" "red"   "blue"

[[5]]
   c1 c2 c3
r1  1  2  3
r2  4  5  6
r3  7  8  9

# the items of a list can be assigned names, then the list can be indexed using item indices or names (with brackets or $)
> names(mylist) <-
c("item1","item2","item3","item4","item5")

> mylist
$item1
[1] 5

$item2
[1] 1 2 3 4 5

$item3
[1] "Hello"

$item4
[1] "green" "red"   "blue"

$item5
   c1 c2 c3
r1  1  2  3
r2  4  5  6
r3  7  8  9

> mylist[["item3"]]
[1] "Hello"
> mylist$item3
[1] "Hello"

# note the differences between indexing using single and double brackets
> mylist["item3"]
$item3
[1] "Hello"
> class(mylist["item3"])
[1] "list"
> class(mylist[["item3"]])
[1] "character"
```

Data Frames

```
# Data Frame object is a Table combining
columns of different types (numeric,
characters, or logical variables)

> name <- c("Bill", "Chris", "Jane", "Terry",
"John", "Sarah")
> sex <- c("M", "M", "F", "F", "M", "F")
> age <- c(20, 17, 21, 17, 21, 20)
> math <- c(68, 83, 64, 86, 75, 89)
> biology <- c(75, 88, 73, 78, 81, 85)
> chemistry <- c(87, 78, 62, 81, 84, 79)

> df <- data.frame(name, sex, age, math,
biology, chemistry)
> df
   name sex age math biology chemistry
1  Bill   M  20   68      75        87
2 Chris   M  17   83      88        78
3  Jane   F  21   64      73        62
4 Terry   F  17   86      78        81
5  John   M  21   75      81        84
6 Sarah   F  20   89      85        79

# subsetting
> df[1:2,]
   name sex age math biology chemistry
1  Bill   M  20   68      75        87
2 Chris   M  17   83      88        78
> df[,1:2]
   name sex
1  Bill   M
2 Chris   M
3  Jane   F
4 Terry   F
5  John   M
6 Sarah   F
> df[1:2,1:2]
   name sex
1  Bill   M
2 Chris   M

> df[c(1,5),c("name","math","biology")]
  name math biology
1 Bill   68      75
5 John   75      81

# You can access one column in a data frame
using the dollar sign $
> df$name[which(df$age < 18)]
[1] "Chris" "Terry"

> df <- cbind(df, "average"=rowMeans(df[,
c("math","biology","chemistry")]))
> df
   name sex age math biology chemistry  average
1  Bill   M  20   68      75        87 76.66667
2 Chris   M  17   83      88        78 83.00000
3  Jane   F  21   64      73        62 66.33333
4 Terry   F  17   86      78        81 81.66667
5  John   M  21   75      81        84 80.00000
6 Sarah   F  20   89      85        79 84.33333
```

Factors

```
> fac <- factor(age)
> fac
[1] 20 17 21 17 21 20
Levels: 17 20 21
> length(fac)
[1] 6
> levels(fac)
[1] "17" "20" "21"
> nlevels(fac)
[1] 3

# aggregate by one factor
> aggregate(df$average, by=list(df$sex), FUN=mean)
  Group.1        x
1       F 77.44444
2       M 79.88889

# aggregate by two factors
> aggregate(df$average, by=list(df$sex, df$age), FUN=mean)
  Group.1 Group.2        x
1       F      17 81.66667
2       M      17 83.00000
3       F      20 84.33333
4       M      20 76.66667
5       F      21 66.33333
6       M      21 80.00000
```

Function apply

```
# Apply some function to specific margins of a matrix or array

# with standard functions
# For matrices, MARGIN=1 for rows and MARGIN=2 for columns

> apply(mat1, MARGIN=1, FUN=max)
r1 r2 r3
 3  6  9

> apply(mat1, MARGIN=2, FUN=median)
c1 c2 c3
 4  5  6

> apply(z, MARGIN=c(1,2), FUN=sum)
   c1 c2
r1 15 21
r2 18 24

# with user-defined function
> apply(mat1, MARGIN=2, FUN=function(x){x*min(x)})
   c1 c2 c3
r1  1  4  9
r2  4 10 18
r3  7 16 27
```

Conditional statements

- **if(condition) statement**

```
> if(mean(mat1) > 3) print("yes")
[1] "yes"
```

- **if(condition) {**

  multiple statements

  }

```
> if(mean(mat1) > 3)
        {
        print("mean of mat1 is", quote=FALSE)
        print(mean(mat1))
        }
[1] mean of mat1 is
[1] 5
```

- **if(condition) statement1 else statement2**

```
if(mean(mat1) > 10) print("yes") else print("no")
[1] "no"
```

- **If(condition) {**

  multiple statements

  } else {multiple statements

  }

- **ifelse(condition, yes, no)**

```
ifelse(mean(mat1) > 3, "yes", "no")
[1] "yes"
ifelse(mean(mat1) > 3, max(mat1), min(mat1))
[1] 9
```

- **Switch(expr, statement1, statement2, …)** expr is integer or character

```
fun <- "sum"
switch(fun, "mean"=mean(mat1), "median"=median(mat1),
"sum"=sum(mat1))
[1] 45
```

- **if … else ladder**

```
if(condition1)
{
statement1
} else if (condition2)
    {
    statement2
    } else if (condition3)
        {
        statement3
        } else {
        statement4
        }
```

e.g.

```
> MyNumbers <- c(-10, -5, 0, 1, 3)
> if (max(MyNumbers) > 10)
   {
   print("max is greater than 10")
   } else if (max(MyNumbers) > 5)
      {
      print("max is greater than 5")
      } else if (max(MyNumbers) >= 0)
         {
         print("max is not negative")
         } else {
                print("max is negative")
                }
[1] "max is not negative"
```

- **Nested if statements**

```
if (condition1)
   {
   if (condition2)
      {
       statement1
       } else {
       statement2
      }
  } else {
  statement3
  }
```

e.g.

```
> MyNumbers <- c(-10, -5, 0, 1, 6)
> if (ncol(MyNumbers)==3)
     {
     if (nrow(MyNumbers)==3)
        {
        print("mat1 is a 3-by-3 matrix")
        } else {
        print("mat1 does not have 3 rows")
        }
     } else {
     print("mat1 does not have 3 columns")
     }
[1] "mat1 does not have 3 rows"
```

Multiple conditions with Logical operators

AND (&)

```
> if(is.matrix(mat1) & all(mat1 >= 0)) print("mat1 is a non-
negative matrix")
[1] "mat1 is a non-negative matrix"
# is.matrix(x) return TRUE if x is a matrix object
# all(x) return TRUE if all elements of the logical vector/matrix
x are TRUE
```

OR (|)

```
> if(any(mat1%%2 > 0) | any(mat1 < 0)) print("mat1 has at least
one odd number or one negative number")
[1] "mat1 has at least one odd number or one negative number"
# any(x) return TRUE if any element of the logical vector/matrix
x is TRUE
# X %% Y finds the remainder of dividing X by Y
# X %/% Y finds the integer of dividing X by Y
```

NOT (!)

```
> mat1[which(!(mat1%%2 == 0) & (mat1%%3 != 0))]
[1] 1 7 5
```

For() loop

- Repeats statements inside the loop while changing value in steps

```
for(value in sequence)
   {
   statements
   }
```

Example

```
> m <- matrix(0, nrow(mat1), ncol(mat1))
> for(k in 1:length(mat1))
# or for(k in seq(1, length(mat1), by=1))
        {
        s <- sqrt(mat1[k])
        if(floor(s)==s) m[k] <- s else m[k] <- mat1[k]
        }
> m
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    2    5    6
[3,]    7    8    3
```

While() loop

- Repeats statements inside the loop while the condition is satisfied

```
while(condition)
    {
    statements
    }
```

example

```
> i <- 1
> while (i < 6)
      {
      print(i)
      i = i+1
      }
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

Creating User-Defined Functions

- The general syntax of a user-defined function is

```
fun_name <- function(argument1, argument2, …){
statements
return(val)
}
```

Example

```
largestElement <- function(mat1, mat2)
{
#check if matrices have identical dimensions
if(sum(dim(mat1) == dim(mat2)) == 2)
          {
          mat3 <- matrix(0, nrow(mat1), ncol(mat1))
          ind1 <- which(mat1 >= mat2)
          ind2 <- which(mat1 < mat2)
          mat3[ind1] <- mat1[ind1]
          mat3[ind2] <- mat2[ind2]
          return(mat3)
          } else print("The two matrices have different dimensions!")
}
largestElement(mat1, t(mat1))
```

```
> largestElement(mat1, t(mat1))
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    4    5    8
[3,]    7    8    9
```

```
> mat1
   c1 c2 c3
r1  1  2  3
r2  4  5  6
r3  7  8  9

> t(mat1)
   r1 r2 r3
c1  1  4  7
c2  2  5  8
c3  3  6  9
```

```
# an object named 'largestElement' should be in workspace after the user-defined
function is created. Use ls() to see that. It can be deleted from workspace using
rm(largestElement).
```

Save and load objects in R

R provides two file formats of its own for storing data, *.RDS* and *.RData*. RDS files can store a single R object, and RData files can store multiple R objects.

```
# save multiple data objects into one file
> save(a, b, mat1, df, file="selectedData.RData")

# remove all listed objects from the workspace
> rm(list=ls())
# load data objects to the workspace
> load("selectedData.RData")
> ls()
[1] "a"    "b"    "mat1"    "df"

# save all data objects in your workspace
> save.image(file="my_workspace.RData")
```

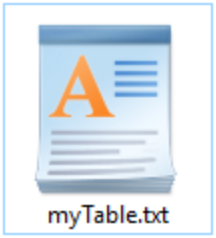
write.table

Saves a data frame as a text file (tab-delimited, comma-delimited, …). You can open such file in Microsoft Excel.

```
write.table(x, file="fileName", append=FALSE, quote=TRUE, sep=" ",
row.names=TRUE, col.names=TRUE)
```

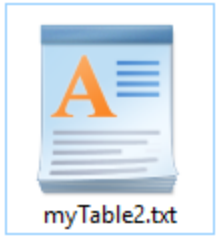
x	the object to be written, preferably a matrix or data frame.
file	a character string naming a file.
append	logical. Only relevant if file is a character string. If TRUE, the output is appended to the file. If FALSE, any existing file of the name is destroyed.
quote	a logical value (TRUE or FALSE) or a numeric vector. If TRUE, any character or factor columns will be surrounded by double quotes. If a numeric vector, its elements are taken as the indices of columns to quote. In both cases, row and column names are quoted if they are written. If FALSE, nothing is quoted.
sep	the field separator string. Values within each row of x are separated by this string.
row.names	either a logical value indicating whether the row names of x are to be written along with x, or a character vector of row names to be written.
col.names	either a logical value indicating whether the column names of x are to be written along with x, or a character vector of column names to be written.

```
> write.table(df, file="myTable.txt", append=FALSE,
quote=FALSE, sep="\t", row.names=FALSE, col.names=TRUE)
```



Name	sex	age	math	biology	chemistry	average
Bill	M	20	68	75	87	76.66667
Chris	M	17	83	88	78	83.00000
Jane	F	21	64	73	62	66.33333
Terry	F	17	86	78	81	81.66667
John	M	21	75	81	84	80.00000
Sarah	F	20	89	85	79	84.33333

```
> write.table(df, file="myTable2.txt", append=FALSE,
quote=TRUE, sep=",", row.names=TRUE, col.names=TRUE)
```




```
"name","gender","age","math","biology","chemistry"
"1","Bill","M",20,68,68,64
"2","Chris","M",17,83,73,67
"3","Jane","F",21,64,72,74
"4","Terry","F",17,86,93,83
"5","John","M",21,75,80,79
```

read.table

```
read.table(file, header=FALSE, sep="", dec = ".", row.names, col.names,
as.is = !stringsAsFactors, skip=0, comment.char = "#", stringsAsFactors =
default.stringsAsFactors(), fileEncoding = "", encoding = "unknown")
```

```
> tab <- read.table("myTable.txt", header=TRUE)
> tab
   name gender age math biology chemistry
1  Bill      M  20   68      68        64
2 Chris      M  17   83      73        67
3  Jane      F  21   64      72        74
4 Terry      F  17   86      93        83
5  John      M  21   75      80        79
> class(tab)
[1] "data.frame"

# use skip parameter to
# skip some lines (e.g.
# comment lines) in
# some text files.
# Example:
```

```
this is file myTable_skip.txt
it has 5 rows
it has 6 columns
% data part starts here
 Name gender   age   math  biology  chemistry
 Bill      M    20     68       68         64
Chris      M    17     83       73         67
 Jane      F    21     64       72         74
Terry      F    17     86       93         83
 John      M    21     75       80         79
```

```
# tab <- read.table("myTable_skip.txt", header=TRUE, skip=4)
```

write.csv & read.csv

```
# csv files are comma-delimited text tables similar to excel
# sheets and can be opened or converted to xlsx format using
# Microsoft excel
```

```
> write.csv(x=df, file="myTable3.csv", row.names=FALSE,
quote=FALSE)
```

```
> write.csv(x=df, file="myTable4.csv",
row.names=c("r1","r2","r3","r4","r5"), quote=TRUE)
```

```
> tab <- read.csv(file="myTable3.csv", header=TRUE)
> tab
   name sex age math biology chemistry  average
1  Bill   M  20   68      75        87 76.66667
2 Chris   M  17   83      88        78 83.00000
3  Jane   F  21   64      73        62 66.33333
4 Terry   F  17   86      78        81 81.66667
5  John   M  21   75      81        84 80.00000
6 Sarah   F  20   89      85        79 84.33333
```

```
> tab <- read.csv(file="myTable4.csv", header=TRUE)
> tab
   X  name sex age math biology chemistry  average
1 r1  Bill   M  20   68      75        87 76.66667
2 r2 Chris   M  17   83      88        78 83.00000
3 r3  Jane   F  21   64      73        62 66.33333
4 r4 Terry   F  17   86      78        81 81.66667
5 r5  John   M  21   75      81        84 80.00000
6 r6 Sarah   F  20   89      85        79 84.33333

> tab <- read.csv(file="myTable4.csv", header=TRUE, row.names=1)
> tab
    name gender age math biology chemistry
r1  Bill      M  20   68      68        64
r2 Chris      M  17   83      73        67
r3  Jane      F  21   64      72        74
r4 Terry      F  17   86      93        83
r5  John      M  21   75      80        79
```

Useful Functions

sort(x, decreasing=FALSE, index.return=FALSE)

```
# sort a vector into ascending or descending order.
```

Examples

```
> sort(c(1,4,8,2,9))
[1] 1 2 4 8 9
> sort(c("d","b","a","c","e"))
[1] "a" "b" "c" "d" "e"

# sort rows in the data frame by student name alphabetically in decreasing order
> s <- sort(x=df$name, decreasing=TRUE, index.return=TRUE)
> class(s)
[1] "list"
> length(s)
[1] 2
> s
$x
[1] "Bill"  "Chris" "Jane"  "John"  "Sarah" "Terry"
$ix
[1] 1 2 3 5 6 4

> df_sorted <- df[s$ix,]
> df_sorted
   name sex age math biology chemistry  average
1  Bill   M  20   68      75        87 76.66667
2 Chris   M  17   83      88        78 83.00000
3  Jane   F  21   64      73        62 66.33333
5  John   M  21   75      81        84 80.00000
6 Sarah   F  20   89      85        79 84.33333
4 Terry   F  17   86      78        81 81.66667
```

grep(pattern, x, ignore.case=FALSE, value=FALSE, invert=FALSE)

```
# search for matches to argument pattern within each element of a character vector.
# Try: > ?grep

# pattern          character string to be matched in the given character vector.
# x                a character vector where matches are sought, or an object   which
                   can be coerced by as.character to a character vector.
# ignore.case      if FALSE, the pattern matching is case sensitive and if
                   TRUE, case is ignored during matching.
# value            If FALSE, a vector containing the (integer) indices of the matches
                   determined by grep is returned, and if TRUE, a vector containing
                   the matching elements themselves is returned.
# invert           If TRUE return indices or values for elements that do not match.
```

Examples

```
> grep(pattern="CT", x=c("TCCG","CCCC","GGCG","GCTA","GCTA"))
[1] 4 5
> grep(pattern="CT", x=c("TCCG","CCCC","GGCG","GCTA","GCTA"), value=TRUE)
[1] "GCTA" "GCTA"
> grep(pattern="M", df$sex)
[1] 1 2 5
```

```
gsub(pattern, replacement, x, ignore.case=FALSE)
# perform replacement of the matches. Try:
# > ?gsub
```

Examples

```
> gsub(pattern="M", replacement="Male", x=df$sex)
[1] "Male"   "Male"   "F"    "F"    "Male"   "F"
```

```
sample(x, size, replace=FALSE)
# takes a random sample of the specified size from the elements of x using
either with or without replacement. To show the help page on the R console
try: > ?sample
```

Examples

```
> df$sex
[1] "M" "M" "F" "F" "M" "F"

# sample with replacement
> sample(x=df$sex, size=8, replace=TRUE)
[1] "M" "M" "F" "M" "M" "M" "F" "M"

# sample without replacement (size must be < length(x))
> sample(x=df$math, size=3, replace=FALSE)
[1] 89 75 64
```